

GoalCoach

Supplementary Understanding, Architecture Clarifications & Feedback

Purpose

This document records our current understanding of the architecture review, clarifies ambiguous implementation points, and identifies concrete decisions that still need to be validated by the team.

Core principle: GoalCoach should not become “agentic” by maximizing the number of LLM calls. Its agentic behavior should come from persistent learner state, adaptive planning, conditional decisions, tool use, and state-dependent workflow.

```
Goal -> Plan -> Teach -> Grade -> Update Learner State -> Re-plan
```

Document Structure

- Part I - Supplementary Understanding & Clarifications
- Part II - Additional Recommendations / TODO
- Part III - Practical Understanding of the Four-Week Sprint

PART I - SUPPLEMENTARY UNDERSTANDING & CLARIFICATIONS

1. Deterministic Does Not Mean “Non-Agentic”

“Deterministic” means that the same relevant system state leads to the same routing decision.

```
review_due = true
↓
route to review-related workflow
```

GoalCoach should use deterministic logic where the decision is already expressible as a reliable rule and reserve LLM reasoning for tasks that genuinely require language understanding, flexible judgment, or generation.

2. Agent $\neq$ LLM Call

An agent does not have to invoke an LLM every time it executes. An agent can use deterministic Python logic, algorithms, database queries, vector retrieval, external tools/APIs, LLM reasoning, or combinations of these.

Component	Core Question	Main Mechanism
Workflow Orchestrator	Who should act next?	Deterministic state machine
Progress & Mastery Agent	What is the learner's current state?	Algorithms + DB + grading evidence
Goal Planning Agent	What should the learner learn/review next?	State + rules + optional LLM reasoning
Retrieval Agent	What learning material should be used?	SQL/filtering + optional vector search
Grader	How well did the learner answer?	LLM + grading rubric
Teaching Agent	How should the concept be explained/practiced?	LLM + retrieved material

3. Progress Agent vs Goal Planning Agent

The current wording creates a potential ambiguity: Goal Planning should not be understood as running only when a goal is first created or modified.

Progress & Mastery Agent

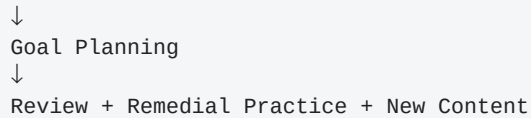
Its primary responsibility is to maintain and expose the learner's current learning state. It answers: **“What is the learner's current state?”**

```
Student answers exercises
↓
Grader
↓
Progress & Mastery Agent
↓
Mastery | Retention | Error Profile | Review Schedule
↓
Learner State
```

Goal Planning Agent

The Goal Planning Agent consumes learner state and determines what should happen next. It answers: **“Given the learner's goal and current state, what should the learner study next?”**

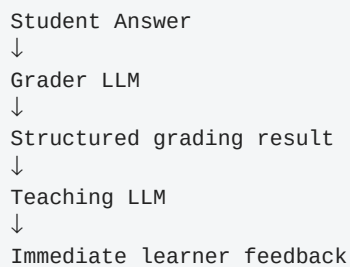
```
Long-term goal: Pass HSK1
+
Current learner state
```



Goal Planning can operate at two levels: long-term planning when a goal is created/changed, and dynamic short-term planning when a session finishes, the daily plan is completed, performance invalidates the plan, or scheduled review must be incorporated.

4. Proposed Runtime Architecture

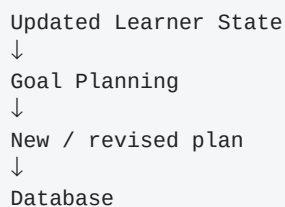
User-facing synchronous path



State-update path



Planning path



The planner therefore does not have to regenerate the entire curriculum after every individual exercise.

5. Understanding the Over-Agentification Bottleneck

The problem is not the existence of several agents. The real problem would be implementing every agent as a sequential LLM call.



The solution is to use LLMs only where language reasoning is needed, keep deterministic state updates deterministic, avoid running every agent for every exercise, run independent operations concurrently where appropriate, move non-urgent work outside the critical response path, and cache reusable retrieval/teaching results.

6. Free-Form Chinese Grading and Rubric-Based Evaluation

GoalCoach cannot simply compare `student_answer == expected_answer` because several different Chinese answers may be valid. Likewise, a single subjective score out of 100 would make learner-state updates unstable.

Dimension	Question
Semantic / Task Achievement	Does the learner express the intended meaning and satisfy the task?
Grammatical Correctness	Is the sentence grammatically valid?
Target-Concept Mastery	Did the learner demonstrate the specific concept being tested?
Word Order	Is Chinese word order correct?
Completeness	Are required sentence components present?
Vocabulary Appropriateness	Is the vocabulary suitable for the intended meaning/context?

Important: do not simply average all dimensions. Target-concept mastery and task achievement should act as candidate gating dimensions.

Task: Use ■ to ask "Are you a student?"
Learner: ■■■■■■

Meaning / Grammar / Word Order = high
Target-Concept Mastery (■) = fail

A simple average could incorrectly mark this as a pass. A first engineering hypothesis is to require minimum scores on target-concept mastery and task achievement, then consider overall quality.

Initial hypothesis only - NOT an HSK official standard:
Target Concept Mastery ≥ 0.70
AND Semantic / Task Achievement ≥ 0.70
AND overall quality ≥ 0.70

A possible mastery interpretation is: ≥ 0.85 mastered/advance; 0.65-0.84 partial/more practice; < 0.65 remedial. These thresholds are provisional and must be validated. Mastery should also be based on evidence across multiple exercises rather than a single answer.

7. Runtime Evaluation vs Offline Evaluation

Runtime grading

Learner Answer -> Grader Model A -> Rubric-based structured result -> Progress + Teaching

A second LLM judge should not automatically be called for every learner answer because this increases cost and latency.

Offline evaluation

Human-labelled HSK answers
↓
Grader Model A
↓
Compare prediction against human label

For a small fixed MVP benchmark, Human vs Grader A is sufficient. Model B becomes useful mainly as evaluation workload scales: larger/new test sets, regression testing after model/prompt/rubric changes, and disagreement/error analysis.

Human labels = benchmark anchor. Model B = optional scalable evaluator/error analyst, not ground truth.

8. Normal Database vs Vector Database

A relational database such as SQLite stores structured learner state and structured curriculum information: user, goal, mastery, retention, error profile, review schedule, sessions, concept_id, HSK level, vocabulary, grammar point, prerequisites, curriculum order and topic.

A vector database such as ChromaDB supports semantic retrieval where the system does not already know the exact record it needs.

SQL/structured retrieval – we know the exact concept/data we want. Vector retrieval = we know the semantic learning need but must discover the most relevant material.

Concrete Trigger Conditions for Vector Retrieval

Retrieval Case	Example	Expected Method
Exact concept	concept_id = hsk1_grammar_ma	SQL / exact lookup
Structured constraints	HSK1 + ■ + exercise	SQL / metadata filter
Semantic learner need	Understands questions but repeatedly misplaces ■	Vector candidate
Unknown / unlabelled error	Overuses ■ before verbal predicates	Vector candidate
Multiple related concepts	Confusion involving ■ / ■ / aspect	Vector candidate
Large corpus after filtering	Many plausible materials remain after metadata filtering	Metadata filter + vector ranking

```
Retrieval Request
↓
Exact concept / structured filters known?
■■ YES -> SQL / structured retrieval
■■ NO
↓
Semantic / ambiguous need OR many candidates?
■■ YES -> metadata filter -> vector search -> top-k -> learner-specific re-ranking
■■ NO -> structured retrieval
```

For a small HSK1 MVP, structured retrieval may solve most cases. Vector RAG should be added only if semantic retrieval provides demonstrable value.

9. Workflow: State Machine vs LangGraph

This should not be framed as State Machine vs LangGraph. GoalCoach's architecture is fundamentally a state machine. The implementation choice is pure Python state-machine logic versus using LangGraph to implement the stateful graph/workflow.

State-machine architecture



Pure Python LangGraph
implementation implementation

10. Tool Roles - Avoid Mixing Different Layers

Layer	Purpose	Candidate
Workflow	Route execution based on state	Python State Machine / LangGraph
Data schema	Enforce structured interfaces	Pydantic
Backend/API	Expose GoalCoach to frontend	FastAPI

Layer	Purpose	Candidate
Structured persistence	Store learner/curriculum data	SQLite
Semantic retrieval	Search teaching content by meaning	ChromaDB
LLM access	Access hosted foundation models	OpenRouter / Amazon Bedrock
Local model runtime	Run models on local hardware	Ollama
Prototype UI	User interface	Streamlit

11. OpenRouter vs Amazon Bedrock

OpenRouter is primarily a multi-model API gateway and is attractive for rapid model experimentation and a provider-agnostic MVP.

Amazon Bedrock is a managed generative-AI platform within AWS. It can provide model access while also integrating broader AWS capabilities such as evaluation, security/governance and managed AI services.

The choice should be driven by project goals and discussed with the mentor rather than inherited automatically from the review.

12. Current Minimal Architecture Hypothesis

```
Frontend
↓
FastAPI
↓
Python State Machine
↓
Goal Planning / Progress / Teaching-Grading / Retrieval
↓
Pydantic interfaces
↓
SQLite
↓
One hosted LLM API
```

Additional technology should be introduced when a concrete requirement appears: LangGraph for workflow complexity/observability, ChromaDB for demonstrated semantic retrieval value, Model B for scalable automated evaluation, and Bedrock when AWS-managed capabilities are an explicit project goal.

PART II - ADDITIONAL RECOMMENDATIONS / TODO

TODO 1 - Correct Goal Planning Trigger Definition

Goal Planning should be activated when a learning goal is created or changed, and whenever the current learning plan needs to be regenerated. It should use mastery, retention, error profile and review schedule to balance review, remedial practice and new learning.

TODO 2 - Formalize Progress vs Planning Interfaces

Define explicit inputs/outputs. Progress exposes concept mastery, retention, error profile, review_due and performance history. Goal Planning consumes those plus goal, curriculum and available study time and outputs a DailyPlan.

TODO 3 - Decide Planning Frequency

Determine whether re-planning happens after every exercise, after a concept block, after a session, once per day, or only after significant state change. Initial recommendation: update Progress after exercises; perform major planning at session boundaries or when the current plan becomes invalid.

TODO 4 - Define Which Agent Operations Actually Require an LLM

Likely LLM tasks: free-form grading, natural-language feedback, adaptive explanations and potentially complex planning. Likely deterministic tasks: retention calculation, mastery aggregation, review-date calculation, database retrieval, simple routing and progress calculation.

TODO 5 - Build and Validate a Formal Grading Rubric

Use semantic/task achievement, grammatical correctness, target-concept mastery, word order, completeness and vocabulary appropriateness. Do not simply average them. Candidate gates and provisional thresholds must be validated against learning objectives, pedagogy, products and learner data.

TODO 6 - Build Human-Labeled Evaluation Set Before Judge Model B

Create an HSK1 benchmark of approximately 30-50 representative learner answers. Human labels are the primary benchmark. For the MVP, Human vs Grader A is sufficient. Add Model B only for scalable automated evaluation, regression testing or disagreement/error analysis.

TODO 7 - Validate Whether Vector RAG Is Actually Necessary

Benchmark three retrieval classes: exact concept lookup -> SQL; structured constraints -> SQL/metadata filtering; semantic/ambiguous learner need -> vector candidate. Add vector retrieval only if it demonstrates measurable value.

TODO 8 - Research and Select HSK1 Data Sources

Identify authoritative HSK1 vocabulary, grammar, examples, exercises, topics, potentially audio and common learner errors. Verify license, redistribution rights, structure, quality, HSK version and suitability for SQL/vector retrieval.

TODO 9 - Decide State Machine vs LangGraph After Building the State Model

First define LearnerState, DailyPlan, GradingResult, ProgressUpdate and RetrievalRequest. Prototype routing using plain Python. Adopt LangGraph only if it meaningfully improves implementation, persistence, debugging or observability.

TODO 10 - Decide OpenRouter vs Bedrock with Mentor

Ask whether the project should remain provider-agnostic for rapid model experimentation or whether AWS/Bedrock integration provides specific learning, deployment or evaluation value expected from the program.

TODO 11 - Make the MVP Scope Explicit

HSK1 is the implementation scope used to validate the complete adaptive learning loop. The architecture remains extensible to higher HSK levels. The success criterion is Plan -> Teach -> Grade -> Update -> Adapt -> Re-plan.

PART III - PRACTICAL UNDERSTANDING OF THE FOUR-WEEK SPRINT

The four weeks are easier to understand as four capability milestones rather than four groups of technologies.

Week	Simple Question	Meaning
W1	Can it run?	Build a simple but complete learning loop.
W2	Can it adapt?	Grading and learner state must actually change the next plan.
W3	Can it work as an app?	Integrate core logic, persistence, API and frontend.
W4	Can we prove it?	Testing, AI evaluation, hardening and demo.

Week 1 - Run the Minimum Learning Loop

Week 1 is not about making every component intelligent. It is about proving that the complete pipeline can run from beginning to end.

```

Goal = HSK1
↓
Simple Planner chooses ■
↓
Teaching gives one exercise
↓
Learner answers
↓
Simple Grader
↓
Progress updates mastery
↓
SQLite saves LearnerState

```

The Planner, Grader and Progress logic can still be very simple. The purpose is to expose interface problems early. A complete but primitive vertical slice is more useful than several disconnected modules.

W1 = Loop exists. It can be simple, but it must be complete.

Week 2 - Make the Loop Adaptive

Week 2 upgrades the primitive loop into GoalCoach's core adaptive behavior. The Grader should identify what went wrong, Progress should update the relevant learner state, and Goal Planning should use that state to change what comes next.

```

Learner: ■■■■
↓
Grader: target concept ■ failed / missing_■
↓
Progress: ■ mastery ↓ + error profile updated
↓
Goal Planning reads new state
↓
Next plan includes remedial practice for ■

```

The clearest proof is to give two learners the same HSK1 goal but different mastery/error states and show that they receive different next plans.

W2 = Loop becomes intelligent/adaptive.

Week 3 - Integration: Turn the Core into a Usable App

Week 3 can be described as connecting the frontend, but it is broader than frontend work. It is the integration stage: the validated core logic, database persistence, API and user interface are connected into one usable system.

```
Frontend / Streamlit
↓ HTTP
FastAPI
↓
GoalCoach Core
↓
SQLite
```

The critical test is multi-session persistence: close the application, reopen it later, recover yesterday's learner state, and let that state influence today's plan.

W3 = Backend + DB + API + Frontend integration.

Week 4 - Testing, Evaluation, Hardening and Demo

Week 4 is testing, but not only software testing. It includes engineering edge cases, AI evaluation against human ground truth, error analysis, bug fixing, prompt/rubric tuning, reliability work and a polished demo.

```
Human Ground Truth
↓
GoalCoach Grader A
↓
Agreement / metrics
↓
Error analysis
```

The demo should focus on behavior change: GoalCoach remembers a learner error, updates mastery/error state, changes the next plan, and later schedules review as retention changes.

W4 = We can prove it works reliably.

Four-Week Plan in One View

```
W1 - It runs.
W2 - It adapts.
W3 - It works end-to-end as an app.
W4 - We can prove it.
```

Do not reduce W1 to “backend” and W3 to “frontend.” W1 already needs a complete core backend learning loop. W3 turns the validated core into a complete usable application.

GoalCoach

补充理解、架构澄清与反馈

目的

本文档记录我们目前对架构评审的理解，澄清其中容易产生歧义的实现问题，并列出具团队仍需进一步验证的具体决策。

核心原则：GoalCoach 不应该通过增加 LLM 调用次数来显得“Agentic”。它真正的 Agentic 行为应来自持续保存的 learner state、自适应规划、条件决策、工具使用，以及由状态驱动的 workflow。

目标 -> 规划 -> 教学 -> 评分 -> 更新学习者状态 -> 重新规划

文档结构

- 第一部分 - 补充理解与架构澄清
- 第二部分 - 额外建议 / TODO
- 第三部分 - 四周开发计划的实际理解

第一部分 - 补充理解与架构澄清

1. Deterministic 不等于 “非 Agentic ”

Deterministic 的意思是：在相关系统状态相同的情况下，系统会做出相同的 routing decision。

```
review_due = true
↓
进入复习相关 workflow
```

当一个决策已经可以通过可靠规则表达时，GoalCoach 应优先使用 deterministic logic；只有真正需要语言理解、灵活判断或生成能力的任务，才交给 LLM reasoning。

2. Agent 不等于一次 LLM Call

一个 Agent 并不需要每次执行都调用 LLM。Agent 可以使用 deterministic Python logic、算法、数据库查询、vector retrieval、外部工具/API、LLM reasoning，或者这些方式的组合。

组件	核心问题	主要机制
Workflow Orchestrator	下一步应该由谁执行？	Deterministic state machine
Progress & Mastery Agent	学习者现在处于什么状态？	算法 + DB + grading evidence
Goal Planning Agent	接下来应该学什么/复习什么？	状态 + 规则 + 可选 LLM reasoning
Retrieval Agent	应该使用什么学习材料？	SQL/filtering + 可选 vector search
Grader	学习者回答得怎么样？	LLM + grading rubric
Teaching Agent	应该怎样解释和练习？	LLM + retrieved material

3. Progress Agent 与 Goal Planning Agent 的区别

目前的表述可能造成一个歧义：Goal Planning 不应该被理解为只在用户第一次创建或修改 goal 时运行。

Progress & Mastery Agent

它的主要职责是维护并暴露学习者当前的学习状态。它回答：“学习者现在是什么状态？”

```
学习者完成练习
↓
Grader
↓
Progress & Mastery Agent
↓
Mastery | Retention | Error Profile | Review Schedule
↓
Learner State
```

Goal Planning Agent

Goal Planning 读取 learner state

并决定下一步做什么。它回答：“考虑学习者的目标和当前状态，下一步应该学什么？”

```

长期目标：通过 HSK1
+
当前 Learner State
↓
Goal Planning
↓
复习 + 补救练习 + 新内容
  
```

Goal Planning 可以有两个层级：goal 创建/改变时的长期规划，以及 session 结束、daily plan 完成、表现使原计划失效、或 scheduled review 需要加入时的动态短期规划。

4. 建议的 Runtime Architecture

面向用户的同步路径

```

学习者回答
↓
Grader LLM
↓
结构化 grading result
↓
Teaching LLM
↓
立即反馈给用户
  
```

状态更新路径

```

Grader Output
├──────────────────┤
↓ ↓
Teaching Progress update
↓ ↓
USER Mastery / Retention / Error / Review
↓
Database
  
```

规划路径

```

更新后的 Learner State
↓
Goal Planning
↓
新的 / 修改后的 plan
↓
Database
  
```

因此 Planner 没有必要在每一道题之后重新生成整个 curriculum。

5. 如何理解 Over-Agentification Bottleneck

真正的问题不是系统里有多个 Agent，而是把每个 Agent 都实现成串行 LLM call。

Student -> Grader LLM -> Progress LLM -> Planning LLM -> Retrieval LLM -> Teaching LLM -> Response

解决方式是：只有需要语言 reasoning 的地方才用 LLM；确定性的状态更新继续保持

deterministic；不要每道题都运行所有

Agent；能并行的操作可以并行；非紧急任务移出关键响应路径；可复用的 retrieval/teaching 结果可以缓存。

6. 中文开放式回答的评分与 Rubric

GoalCoach 不能简单使用 `student_answer ==`

`expected_answer`，因为同一道中文题可能存在多个正确表达。同时，只给一个主观的 100 分制总分，也会让 learner-state update 不稳定。

维度	要判断的问题
Semantic / Task Achievement	是否表达了目标含义并完成任务？
Grammatical Correctness	句子语法是否正确？
Target-Concept Mastery	是否真正使用并掌握本题目标知识点？
Word Order	中文语序是否正确？
Completeness	必要的句子成分是否完整？
Vocabulary Appropriateness	词汇是否适合当前含义和语境？

重要：这些维度不能简单取平均。Target-Concept Mastery 和 Task Achievement 应作为候选 gating dimensions。

题目：使用「吗」表达 “Are you a student?”

学习者：你是不是学生？

语义 / 语法 / 语序 = 高

Target-Concept Mastery (吗) = 失败

如果简单平均，系统可能错误地把这个答案判定为通过。第一版工程假设可以要求 Target Concept 和 Task Achievement 先达到最低门槛，再考虑整体质量。

仅为初始工程假设 - 不是 HSK 官方标准：

Target Concept Mastery ≥ 0.70

AND Semantic / Task Achievement ≥ 0.70

AND overall quality ≥ 0.70

一种候选 mastery 分层可以是： ≥ 0.85 mastered/advance； $0.65-0.84$ partial/more practice； < 0.65 remedial。但这些阈值都必须验证。Mastery 也不应该只根据一次回答决定，而应基于多次练习证据。

7. Runtime Evaluation 与 Offline Evaluation

Runtime grading

Learner Answer -> Grader Model A -> Rubric structured result -> Progress + Teaching

不应该默认每道题都再调用第二个 LLM Judge，因为这会增加 latency 和 cost。

Offline evaluation

人工标注的 HSK answers
↓
Grader Model A
↓
与 Human Label 比较

对于一个固定的小规模 MVP benchmark，Human vs Grader A 已经足够。Model B 主要在 evaluation 规模扩大以后才有意义，例如更多/新的测试集、模型或 prompt/rubric 修改后的 regression testing，以及 disagreement/error analysis。

Human labels = benchmark anchor。Model B = 可选的可扩展 evaluator/error analyst，而不是 ground truth。

8. 普通数据库与 Vector Database

SQLite 这样的 relational database 用来保存结构化 learner state 和结构化 curriculum 信息，例如 user、goal、mastery、retention、error profile、review schedule、session、concept_id、HSK level、vocabulary、grammar point、prerequisites、curriculum order 和 topic。

ChromaDB 这样的 vector database 主要解决 semantic retrieval：系统知道学习需求的语义，但并不知道应该精确读取哪一条记录。

SQL/structured retrieval = 已经知道要找哪个 concept/data。Vector retrieval = 知道学习需求，但需要通过语义搜索发现最相关材料。

Vector Retrieval 的具体触发条件

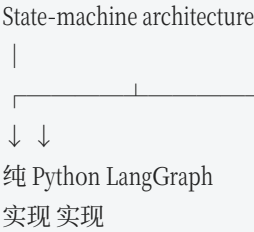
Retrieval 情况	例子	预期方法
精确 concept	concept_id = hsk1_grammar_ma	SQL / exact lookup
结构化条件	HSK1 + 吗 + exercise	SQL / metadata filter
语义学习需求	会提问但反复把「吗」放错位置	Vector candidate
未预定义错误	在动词谓语句前过度使用「是」	Vector candidate
多个相关 concept	混淆了 / 过 / aspect	Vector candidate
过滤后仍有大量候选	metadata filter 后仍有很多可能材料	metadata filter + vector ranking

Retrieval Request
↓
已知 exact concept / structured filters ?
├— 是 -> SQL / structured retrieval
└— 否
↓
是否为 semantic / ambiguous need 或候选过多 ?
├— 是 -> metadata filter -> vector search -> top-k -> learner-specific re-ranking
└— 否 -> structured retrieval

对于小规模 HSK1 MVP，structured retrieval 很可能已经能解决大多数情况。只有 semantic retrieval 证明有实际价值时，才加入 Vector RAG。

9. Workflow：State Machine 与 LangGraph

这不应该被理解为 State Machine 和 LangGraph 二选一。GoalCoach 的架构本质上就是 state machine；真正的选择是用纯 Python 实现，还是用 LangGraph 来实现这个 stateful graph/workflow。



10. 不同工具属于不同层

层	用途	候选工具
Workflow	根据 state routing execution	Python State Machine / LangGraph
Data schema	约束结构化接口	Pydantic
Backend/API	把 GoalCoach 暴露给 frontend	FastAPI
Structured persistence	保存 learner/curriculum data	SQLite
Semantic retrieval	按语义搜索教学内容	ChromaDB
LLM access	访问 hosted foundation models	OpenRouter / Amazon Bedrock
Local model runtime	本地运行模型	Ollama
Prototype UI	用户界面	Streamlit

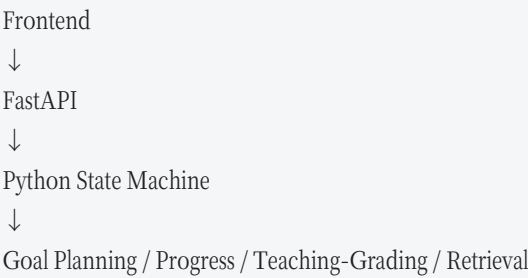
11. OpenRouter 与 Amazon Bedrock

OpenRouter 主要是 multi-model API gateway，适合快速尝试不同模型，以及保持 MVP 的 provider-agnostic。

Amazon Bedrock 是 AWS 内的 managed generative-AI platform。除了模型访问以外，还可以与 AWS 的 evaluation、security/governance 和其他 managed AI services 集成。

最终选择应该由项目目标决定，并与 mentor 讨论，而不是因为评审里提到了某个工具就自动采用。

12. 当前最小架构假设



↓
Pydantic interfaces
↓
SQLite
↓
一个 hosted LLM API

只有出现明确需求时再增加额外技术：workflow complexity/observability 需要时考虑 LangGraph；semantic retrieval 证明有价值时考虑 ChromaDB；evaluation 需要扩展时考虑 Model B；明确需要 AWS managed capabilities 时考虑 Bedrock。

第二部分 - 额外建议 / TODO

TODO 1 - 修正 Goal Planning 的触发定义

Goal Planning 应在学习目标创建或改变时触发，也应在当前 learning plan 需要重新生成时触发。它应使用 mastery、retention、error profile 和 review schedule 来平衡复习、补救练习和新内容。

TODO 2 - 明确 Progress 与 Planning 的接口

定义清晰的 input/output。Progress 输出 concept mastery、retention、error profile、review_due 和 performance history。Goal Planning 读取这些信息，再结合 goal、curriculum 和可用学习时间，输出 DailyPlan。

TODO 3 - 决定 Planning Frequency

验证 re-planning 应该发生在每道题之后、concept block 之后、session 之后、每天一次，还是只在重要 state change 后。初始建议：练习后更新 Progress；主要 planning 在 session boundary 或当前 plan 失效时进行。

TODO 4 - 明确哪些 Agent 操作真的需要 LLM

可能需要 LLM 的任务：free-form grading、自然语言 feedback、自适应解释，以及可能的复杂 planning。可能保持 deterministic 的任务：retention calculation、mastery aggregation、review-date calculation、database retrieval、简单 routing 和 progress calculation。

TODO 5 - 建立并验证正式 Grading Rubric

候选维度包括 semantic/task achievement、grammatical correctness、target-concept mastery、word order、completeness、vocabulary appropriateness。不要简单平均。候选 gating dimensions 和 provisional thresholds 都必须结合学习目标、教学法、现有产品和 learner data 进一步验证。

TODO 6 - 先建立 Human-Labeled Evaluation Set，再考虑 Judge Model B

建立大约 30-50 个有代表性的 HSK1 learner answers。Human labels 是主要 benchmark。MVP 阶段 Human vs Grader A 已经足够。只有需要 scalable automated evaluation、regression testing 或 disagreement/error analysis 时再加入 Model B。

TODO 7 - 验证 Vector RAG 是否真的必要

至少 benchmark 三类 retrieval：exact concept lookup -> SQL；structured constraints -> SQL/metadata filtering；semantic/ambiguous learner need -> vector candidate。只有 vector retrieval 展现出可量化价值时才加入。

TODO 8 - 研究并选择 HSK1 Data Sources

寻找权威 HSK1 vocabulary、grammar、examples、exercises、topics，以及可能的 audio 和 common learner errors。验证 license、redistribution rights、结构、质量、HSK version，以及是否适合 SQL/vector retrieval。

TODO 9 - 建好 State Model 后再决定 State Machine vs LangGraph

先定义 LearnerState、DailyPlan、GradingResult、ProgressUpdate 和 RetrievalRequest，并用纯 Python prototype routing。只有 LangGraph 对 implementation、persistence、debugging 或 observability 有明显价值时再采用。

TODO 10 - 与 Mentor 决定 OpenRouter vs Bedrock

确认项目更需要 provider-agnostic 的快速模型实验，还是 AWS/Bedrock 能提供项目明确需要的 learning、deployment 或 evaluation value。

TODO 11 - 明确 MVP Scope

HSK1 是用于验证完整 adaptive learning loop 的实现范围。架构仍应可以通过扩展 curriculum 和 learning-material layer 支持更高 HSK level。成功标准是 Plan -> Teach -> Grade -> Update -> Adapt -> Re-plan。

第三部分 - 四周开发计划的实际理解

四周计划最好理解成四个能力里程碑，而不是四组技术工具。

周次	最简单的问题	含义
W1	能跑吗？	做一个简单但完整的 learning loop。
W2	会适应吗？	评分和 learner state 必须真正改变 next plan。
W3	能当 App 用吗？	整合 core logic、persistence、API 和 frontend。
W4	能证明靠谱吗？	Testing、AI evaluation、hardening 和 demo。

第1周 - 跑通最小 Learning Loop

第1周不是要求每个组件都已经很聪明，而是证明整条 pipeline 从头到尾能够跑通。

```
Goal = HSK1
↓
最简 Planner 选择「是」
↓
Teaching 给一道练习
↓
学习者回答
↓
最简 Grader
↓
Progress 更新 mastery
↓
SQLite 保存 LearnerState
```

Planner、Grader 和 Progress 逻辑都可以暂时很简单。重点是尽早暴露 interface problem。一个完整但简单的 vertical slice，比几个互相没有连接的 module 更有价值。

W1 = Loop exists。可以简单，但必须完整。

第2周 - 让 Loop 真正 Adaptive

第2周把第1周的简单闭环升级成 GoalCoach 最核心的 adaptive behavior。Grader 要知道错在哪里，Progress 要更新对应 learner state，Goal Planning 再根据新 state 改变下一步学习内容。

```
学习者：我学生
↓
Grader：目标 concept 「是」 失败 / missing_是
↓
Progress：「是」 mastery ↓ + error profile 更新
↓
Goal Planning 读取新 state
↓
Next plan 加入「是」的 remedial practice
```

最清楚的证明方式是：两个用户目标都为 HSK1，但 mastery/error state 不同，因此得到不同 next plan。

W2 = Loop becomes intelligent/adaptive。

第3周 - Integration：把核心变成真正可用的 App

第3周可以说是“连接前端”，但范围比 frontend 更大。它是 integration stage：把已经验证的 core logic、database persistence、API 和 UI 连接成一个完整可使用的系统。

```
Frontend / Streamlit
↓ HTTP
FastAPI
↓
GoalCoach Core
↓
SQLite
```

关键测试是 multi-session persistence：关闭应用，之后重新打开，能够恢复昨天的 learner state，而且这个 state 会影响今天的 plan。

W3 = Backend + DB + API + Frontend integration。

第4周 - Testing、Evaluation、Hardening 与 Demo

第4周确实主要是测试，但不仅是普通软件测试。它包括 engineering edge cases、与 human ground truth 对比的 AI evaluation、error analysis、bug fixing、prompt/rubric tuning、reliability 和最终 demo。

```
Human Ground Truth
↓
GoalCoach Grader A
↓
Agreement / metrics
↓
Error analysis
```

Demo 的重点应该是行为改变：GoalCoach 记住 learner error，更新 mastery/error state，改变 next plan，并在 retention 变化后安排后续 review。

W4 = 我们能够证明它可靠地工作。

四周计划一眼看懂

```
W1 - It runs.
W2 - It adapts.
W3 - It works end-to-end as an app.
W4 - We can prove it.
```

不要把 W1 简化成“做后端”，也不要将 W3 简化成“做前端”。W1 已经需要完整的核心 backend learning loop；W3 是把前两周已经验证的核心变成完整可使用的应用。